HireFlow

RECRUITMENT & APPLICANT TRACKING SYSTEM

ProEduvate Internship · Round 2 · Full Stack Development

Christopher Jesuraj · Rajalakshmi Institute of Technology, Chennai · Project submission document

LIVE APPLICATION

https://hireflow-ten-rho.vercel.app

Deployed on Vercel (frontend + Express API) with a Neon PostgreSQL database. One-click demo logins on the login page.

GITHUB REPOSITORY

https://github.com/christhecreator56/hireflow

Public repository: frontend, backend, Prisma migrations and seed, tests, Docker setup, documentation.

DOWNLOAD SOURCE CODE (ZIP)

https://github.com/christhecreator56/hireflow/archive/refs/heads/master.zip

Direct download of the complete repository, master branch (about 12 MB).

DOWNLOAD PROJECT REPORT (PDF)

https://github.com/christhecreator56/hireflow/raw/master/docs/HireFlow-Project-Report.pdf

32-page report: problem, architecture, build process, page-by-page walkthrough with screenshots, security, tests, limitations (about 5.6 MB).

CLONE WITH GIT

```
git clone https://github.com/christhecreator56/hireflow.git
```

DOCUMENTATION IN THE REPOSITORY

README (this document)	https://github.com/christhecreator56/hireflow#readme
Codebase workflow	https://github.com/christhecreator56/hireflow/blob/master/docs/WORKFLOW.md
Test workflow	https://github.com/christhecreator56/hireflow/blob/master/docs/TESTING.md
API documentation	https://github.com/christhecreator56/hireflow/blob/master/docs/API.md

Database design & ER diagram	https://github.com/christhecreator56/hireflow/blob/master/docs/DATABASE.md
Build phases & bottleneck analysis	https://github.com/christhecreator56/hireflow/blob/master/docs/PHASES.md
OpenAPI specification (JSON)	https://github.com/christhecreator56/hireflow/blob/master/backend/openapi.json
Screenshots folder	https://github.com/christhecreator56/hireflow/tree/master/docs/screenshots

DEMO ACCOUNTS (SEEDED DEMO DATA)

Role	Email
Employer	employer@acme.test
Recruiter	recruiter@acme.test
Hiring manager	manager@acme.test
Candidate	candidate@acme.test

Demo password for every account: Password123! (shared test credential for evaluation only). The README that follows is reproduced in full from the repository.

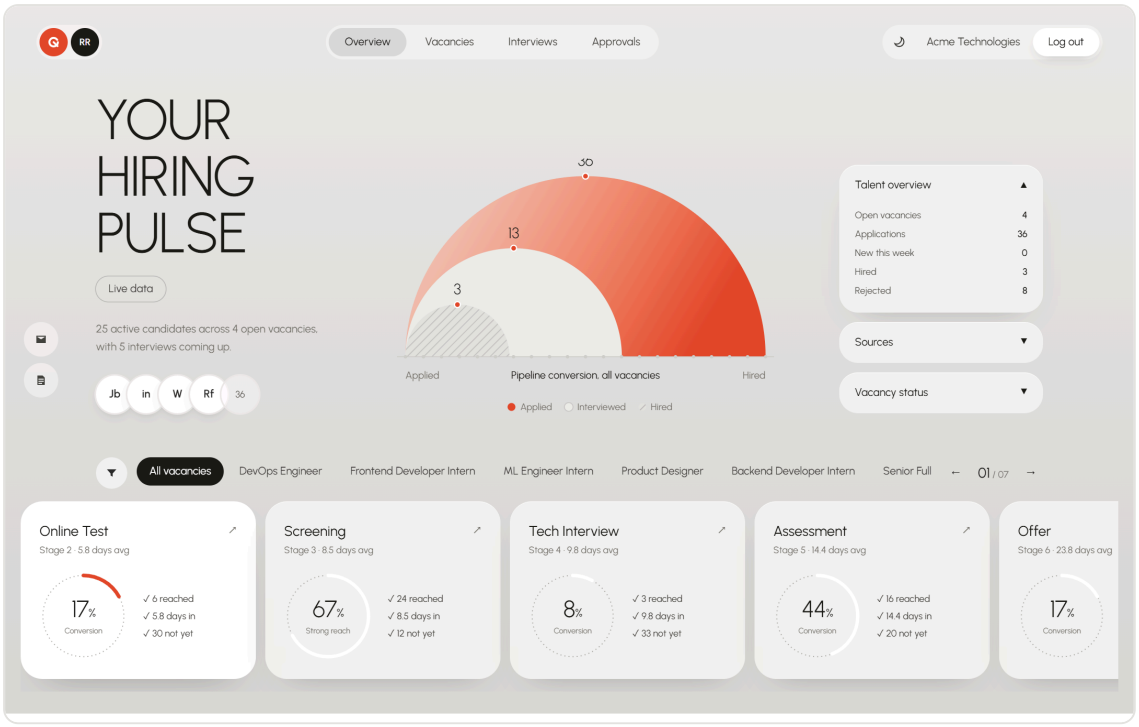
HireFlow

Recruitment & Applicant Tracking System

Publish vacancies, collect applications, run each role through its own configurable hiring pipeline, interview with structured scorecards, notify candidates, and govern candidate data end to end.

React · TypeScript · Tailwind · Node.js / Express · PostgreSQL · Prisma · S3 storage · Docker

Features · Screenshots · Quick start · Architecture · Testing · Docs · Project report (PDF)



Contents

- What it is
- Features
- Screenshots
- Quick start
- Demo accounts
- Architecture
- Tech stack
- Security & privacy
- Testing
- API

- Project structure
- Documentation
- Submission checklist

What it is

HireFlow is a full-stack applicant tracking system built for the **ProEduvate Full Stack Round 2** problem statement. It serves four roles:

Role	What they do
Employer (org admin)	Manages the team, approves offers, sets data-retention policy, handles deletion requests, reads the audit log
Recruiter	Creates and publishes vacancies, designs each role's pipeline, moves candidates, schedules interviews, sends templated emails
Hiring Manager	Sees only their own vacancies, interviews, submits scorecards, approves stage moves gated to them
Candidate	Signs up with consent, uploads a resume (auto-parsed), sees skill-match scores, applies, tracks every stage, exports or deletes their data

Two features go beyond a basic ATS:

1. **Configurable hiring workflow.** Every vacancy owns its own ordered stages. A stage can email the candidate and can require approval from a specific role before anyone enters it. All of this is changed as data, never in code.
2. **Candidate data governance.** Versioned consent capture, role-limited access (interviewers see masked contact details), a retention policy with a nightly sweep, self-service export, and a deletion workflow that anonymizes personal data while keeping the analytics correct.

Features

Area	Highlights
Job management	Create / publish / pause / close vacancies through an enforced status machine; skills, salary range, requirements; custom application questions (text, long text, single/multi choice, yes/no); questions lock after the first application
Candidate portal	Consent-based sign-up, profile, PDF/DOCX resume upload to S3 (5 MB), **resume parsing** (skills, phone, city, LinkedIn, headline), skill-match ring on every job, application timeline, withdraw
Recruitment pipeline	Kanban per vacancy with drag-and-drop (plus a select for touch screens), candidates sorted by skill match, reject with reason, full stage history, optimistic locking so two recruiters can't overwrite each other

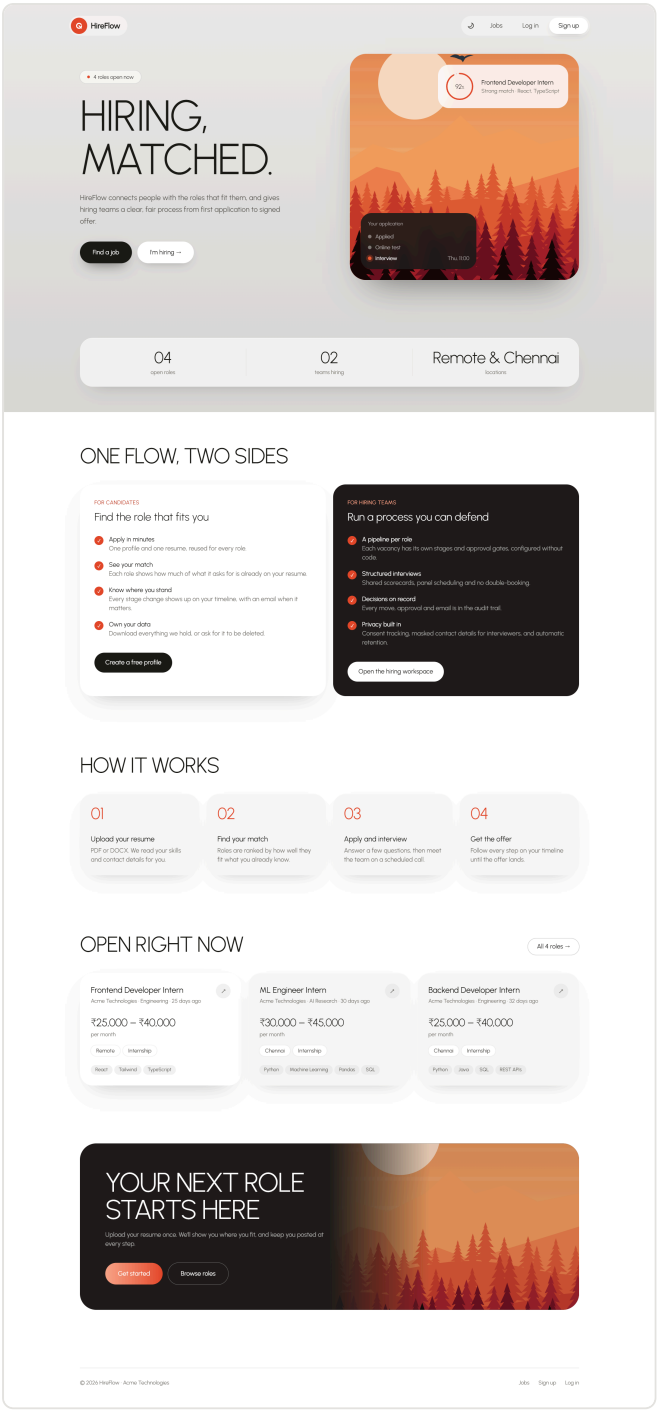
Area	Highlights
Configurable workflow ⭐	Per-vacancy stage editor: rename, reorder, add, delete (blocked while a stage holds candidates), category (active / hired / rejected), notify-candidate + email template, approval gate with approver role; reusable pipeline templates
Interviews	Schedule with a panel, online or onsite; **double-booking prevention**; structured scorecards rated 1–5 per job criterion with a recommendation; interviewer queue with masked PII
Communication	Editable Handlebars templates (`{{candidateName}}` , `{{jobTitle}}` …), sent to a sandbox inbox (Mailpit) without blocking the request; email log; in-app notifications
Reporting	Hiring-pulse dashboard: conversion arc (applied → interviewed → hired), time-to-stage, source of applications, vacancy status, per-stage conversion rings; hiring managers are scoped to their own jobs
Data governance ⭐	Consent records (type, policy version, IP), data export (JSON), deletion request → employer approval → anonymization, retention days + nightly cron + manual run, audit of resume views
Audit history	Append-only log of every write: actor, role, action, entity, before/after JSON, IP; filterable viewer
Experience	Landing page for first-time visitors, full-screen auth, **light & dark themes**, page transitions and data animations, responsive down to 375 px, respects reduced-motion

Screenshots

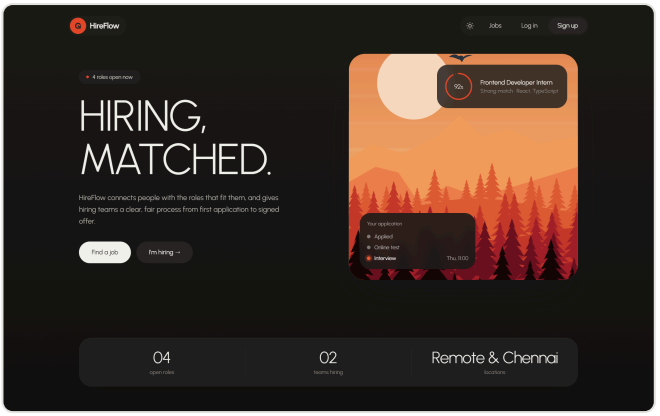
All screenshots use generated demo data. Real resumes are never committed.

Public site

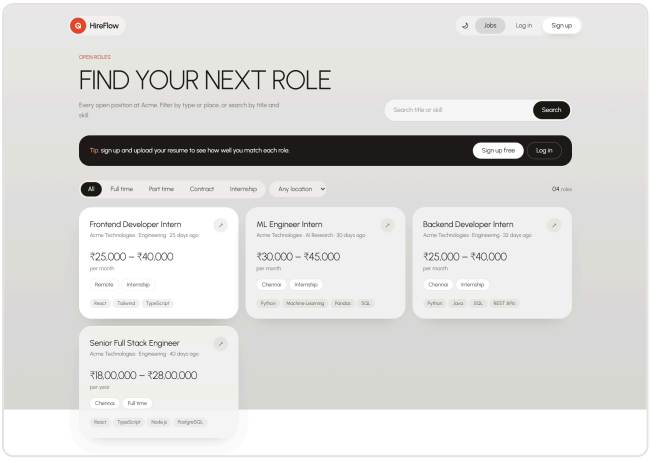
Landing page



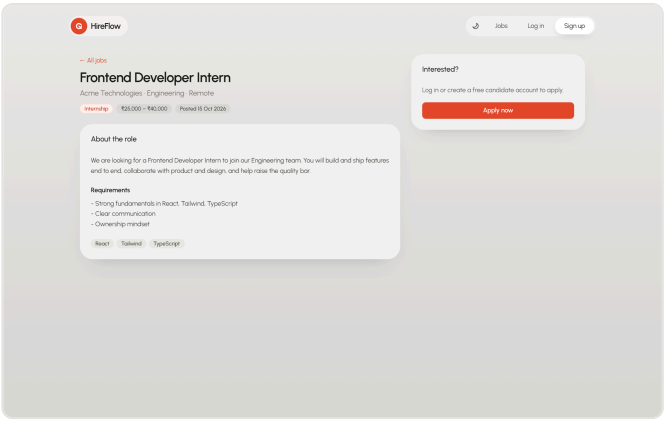
Landing (dark theme)



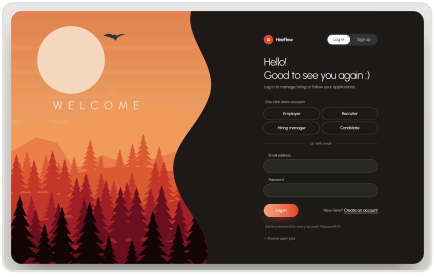
Job board



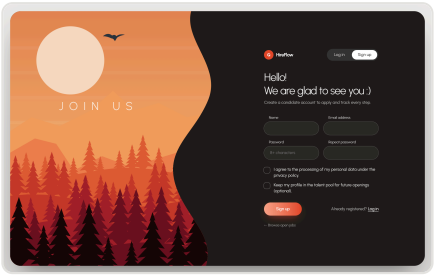
Job detail & application form



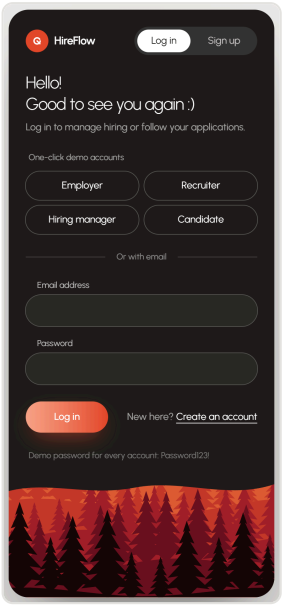
Log in

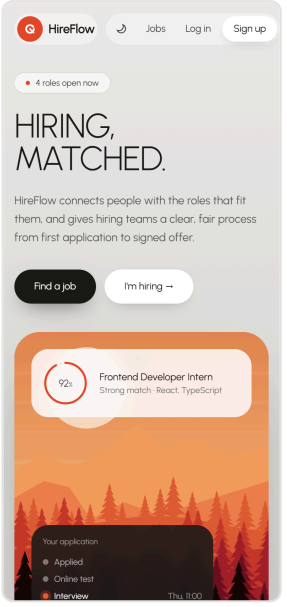


Sign up



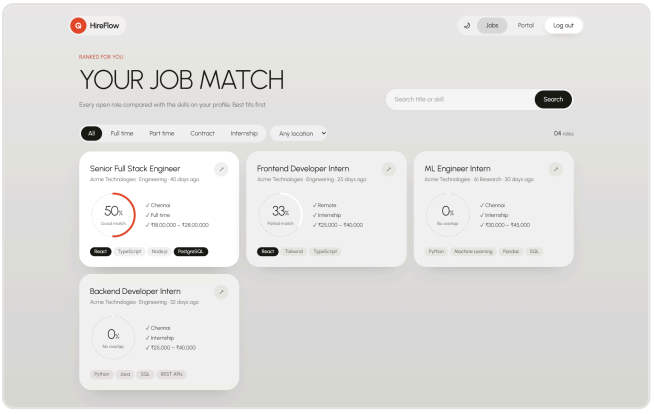
Phone



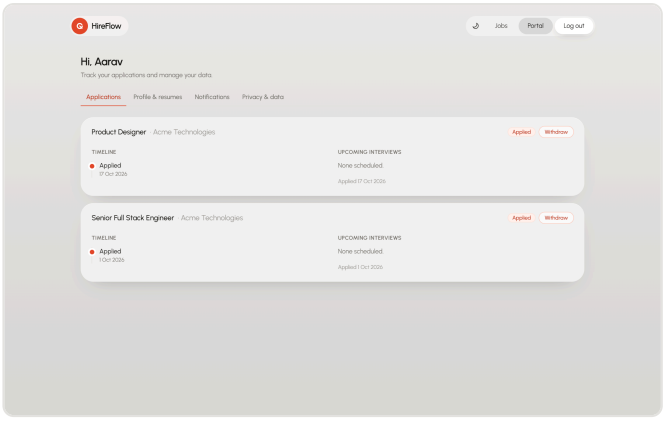


Candidate

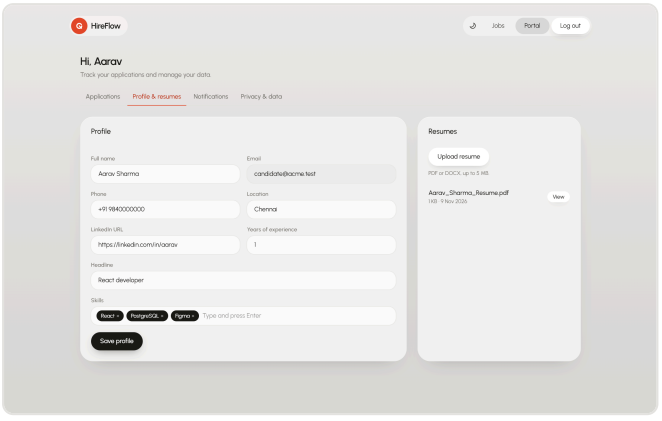
Job match (ranked by your skills)



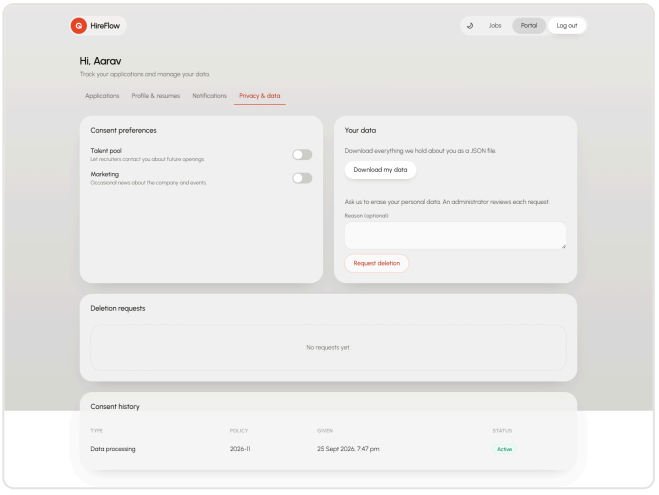
My applications & timeline



Profile & resumes

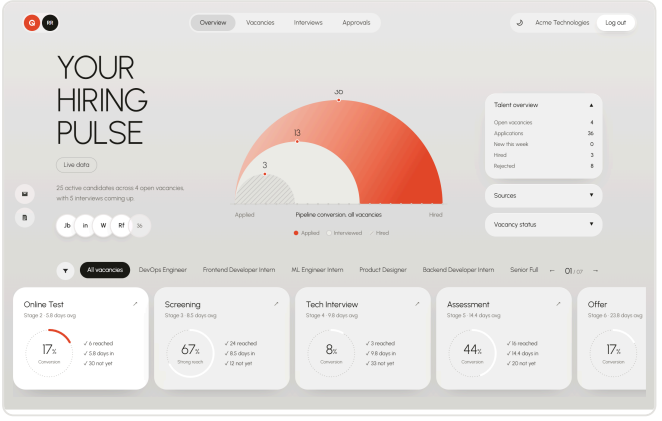


Privacy & data

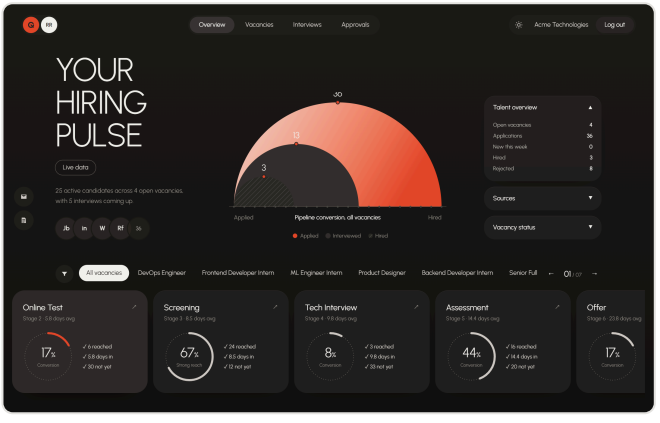


Recruiter / hiring team

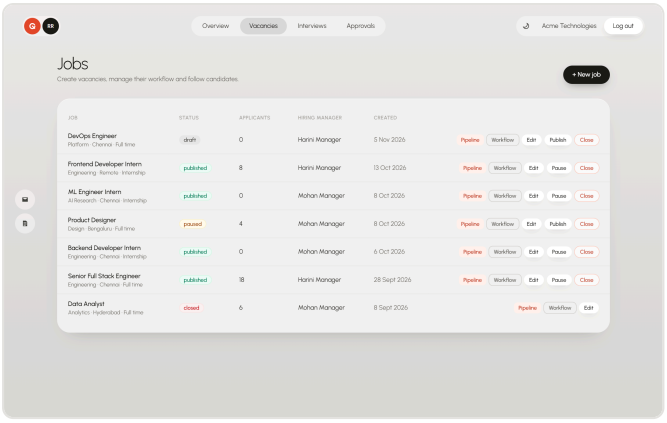
Dashboard



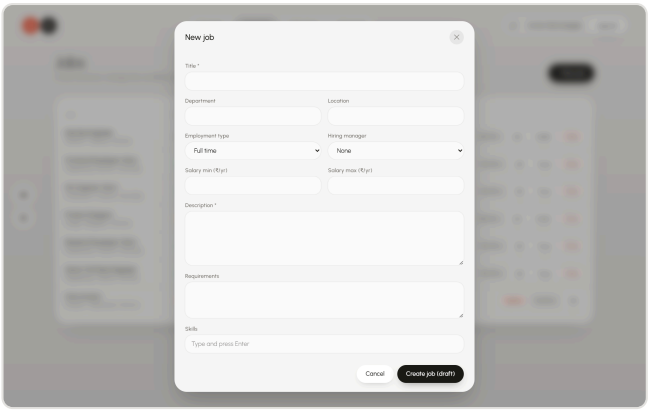
Dashboard (dark)



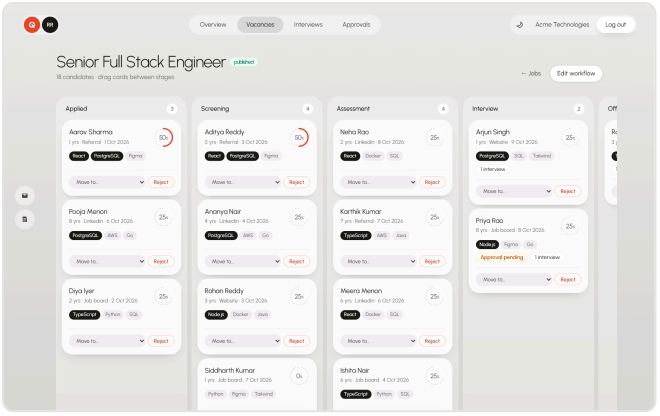
Vacancies



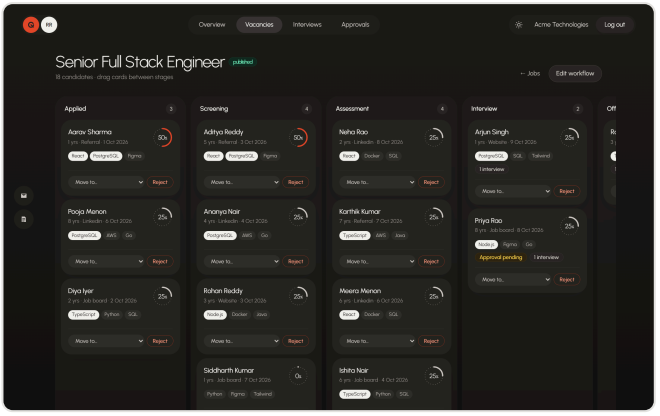
New job form



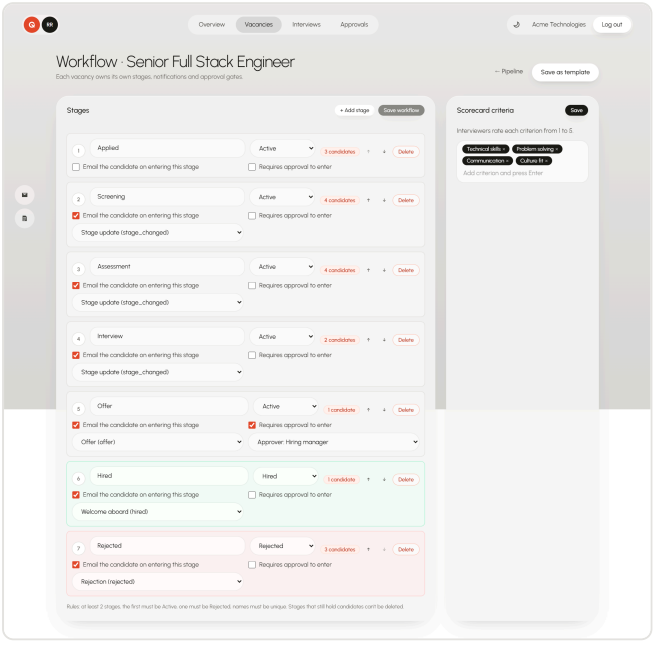
Pipeline Kanban



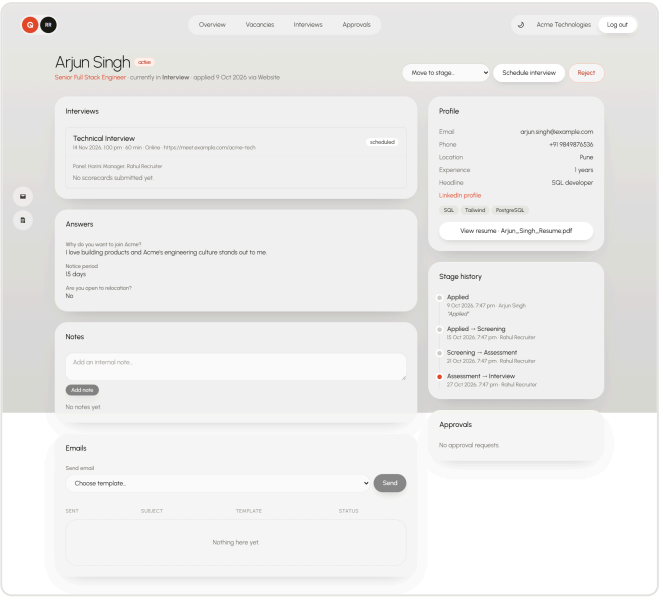
Kanban (dark)



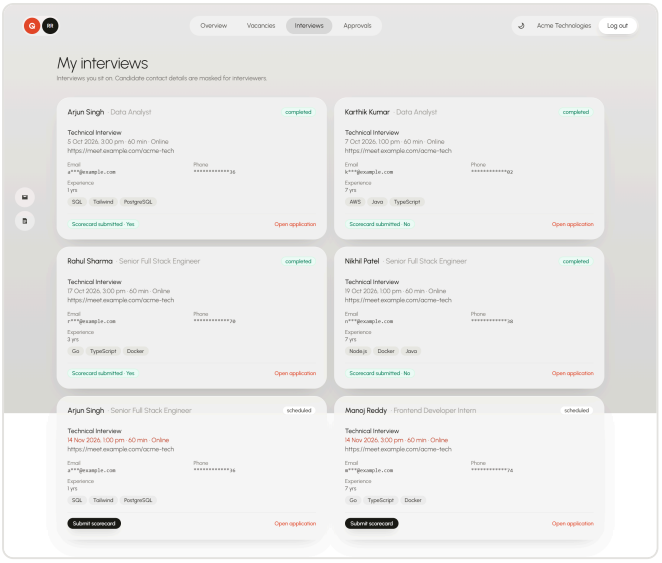
Workflow editor ⭐



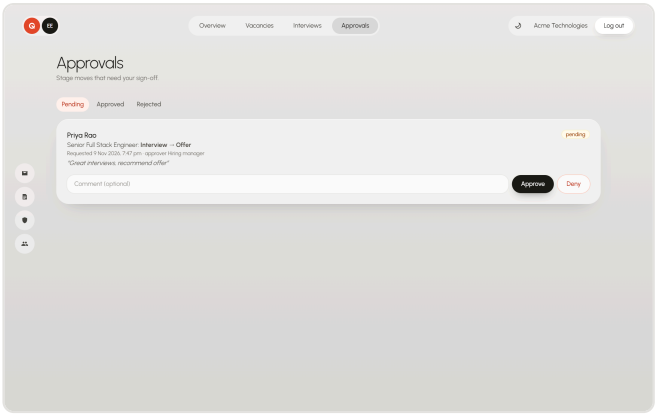
Candidate detail (interviews, scorecards, history)



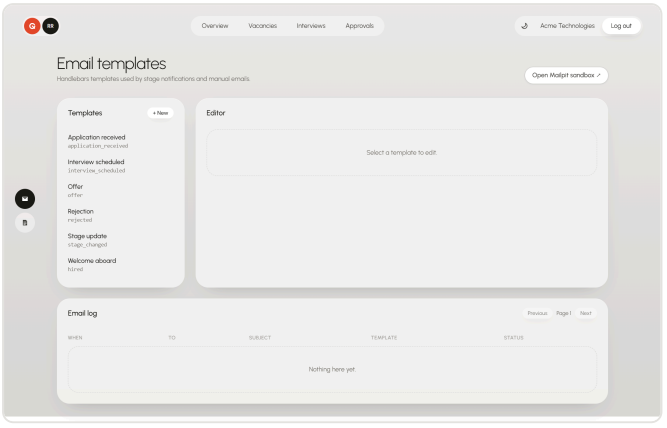
My interviews & scorecards



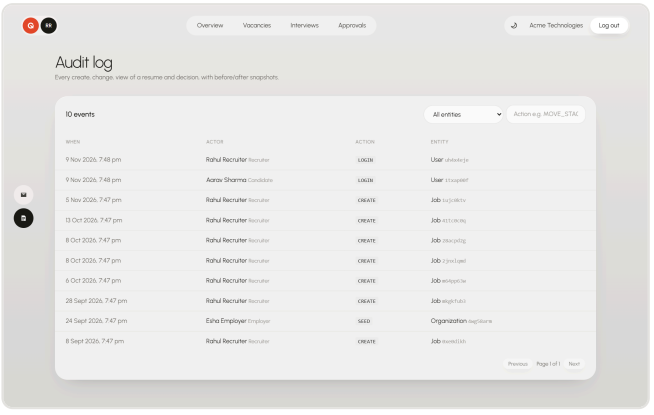
Offer approvals



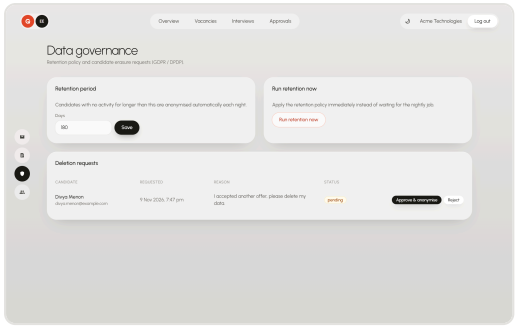
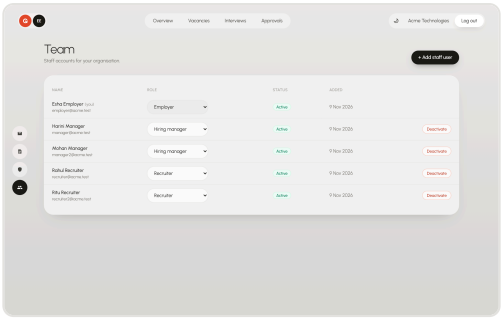
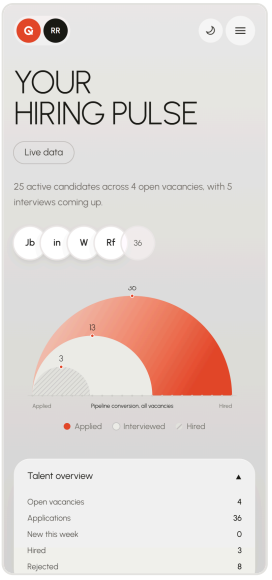
Email templates & log



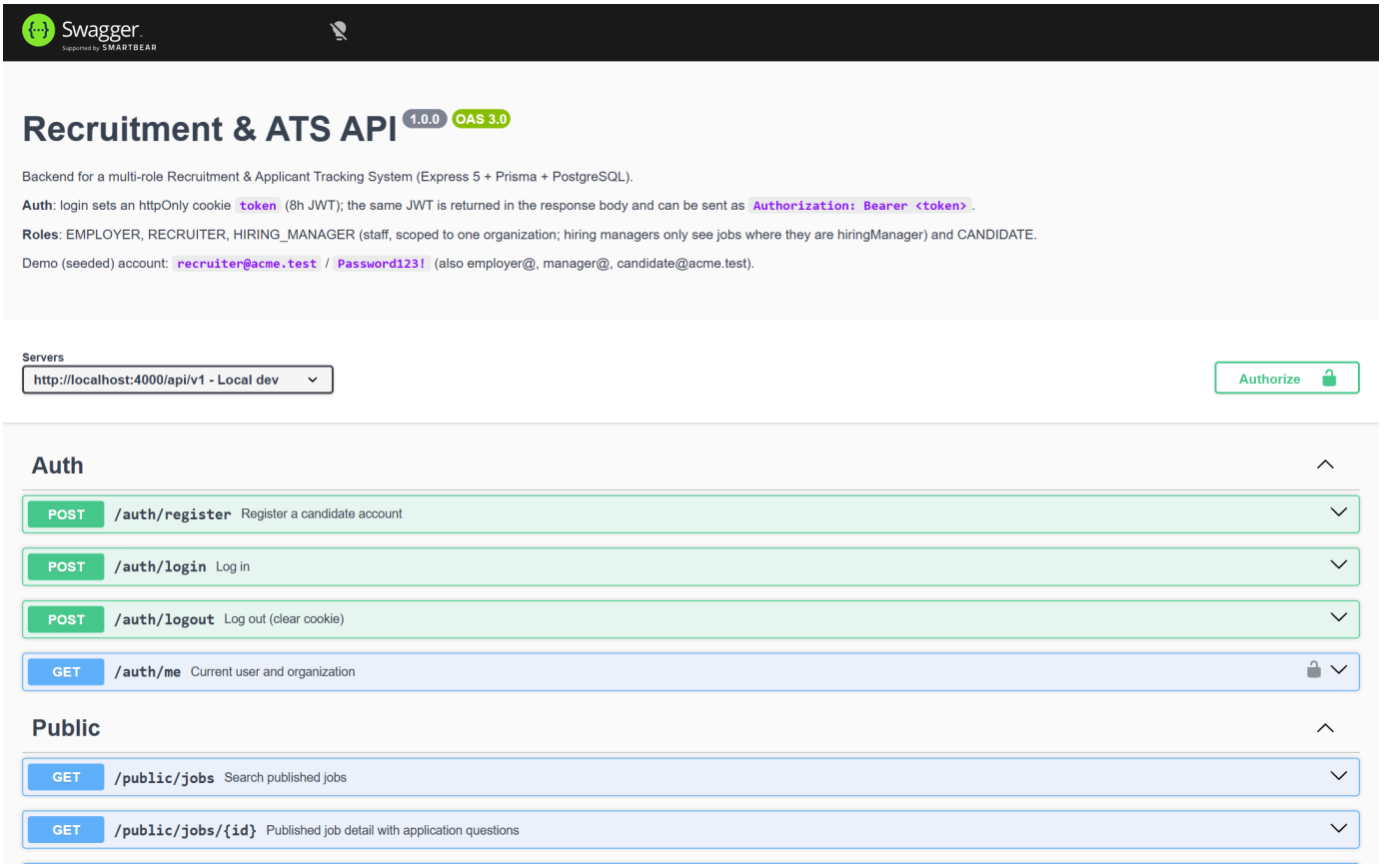
Audit log



Employer

Data governance ⭐	Team & roles	Dashboard on a phone
		

API documentation (Swagger)



Quick start

Requirements: Docker Desktop and Node.js 22.

Everything in Docker

```
docker compose --profile app up -d --build
```

```
docker compose exec api npm run db:seed
```

URL	What
http://localhost:5173	Web app
http://localhost:4000/api/docs	Swagger API docs
http://localhost:8025	Mailpit, the sandbox inbox for all emails

Local development

```
docker compose up -d db storage mailpit
```

```
cd backend && cp .env.example .env && npm install && npx prisma migrate deploy && npm run db:seed &&
npm run dev
```

```
cd frontend && npm install && npm run dev
```

Set `JWT_SECRET` in `backend/.env` to any random string of 16+ characters. The frontend dev server proxies `/api` to `:4000`.

Demo data from real resumes (optional)

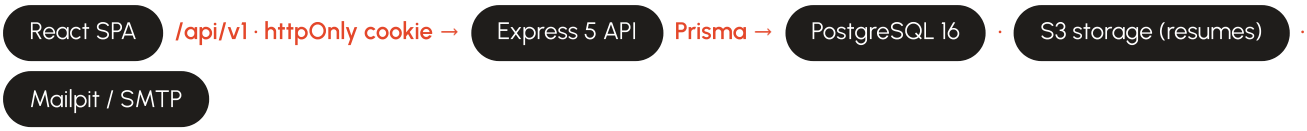
Put PDF resumes in `import/resumes/` (gitignored) and run `npm run db:seed` again. Every resume becomes a candidate: its text is parsed, the original PDF is stored, and the candidate applies to the open job that best fits their skills. They are then placed in that job's pipeline by match (hired, offer awaiting approval, interviewing with scorecards, assessment, rejected). Set `SEED_FAKE=1` to use generated candidates instead.

Demo accounts

Password for every account: **`Password123!`** (seeded demo data). On the login page each role is a one-click button.

Role	Email
Employer	`employer@acme.test`
Recruiter	`recruiter@acme.test`
Hiring manager	`manager@acme.test` (also `manager2@acme.test`)
Candidate	`candidate@acme.test`

Architecture



- **Thin routes, rules in `lib/`.** Each route parses its input with Zod, authorizes, calls the domain logic, writes to the audit log and responds.
- **One path for every stage change.** `moveApplication()` checks the job, the approval gate and the optimistic lock, writes a `StageTransition`, then notifies the candidate.
- **One place for permissions.** `lib/access.ts` decides who may see a job or an application, which is what blocks IDOR (users opening other people's records by changing an ID).
- **Analytics from history.** `StageTransition` is append-only, and every report is a single SQL aggregate.

Full diagrams: docs/WORKFLOW.md · Database & ER diagram: docs/DATABASE.md

Tech stack

Layer	Choice
Frontend	React 19, TypeScript, Vite, Tailwind CSS v4, React Router; custom component kit, SVG charts and match rings
Backend	Node.js 22, Express 5, TypeScript, Zod validation, Multer uploads, node-cron
Database	PostgreSQL 16 with Prisma ORM, migrations and seed
Files	S3-compatible object storage (SeaweedFS locally; AWS S3 or R2 in production) via AWS SDK v3, presigned URLs
Email	Nodemailer + Handlebars templates → Mailpit sandbox
Auth	bcrypt, JWT in httpOnly SameSite cookie (Bearer also accepted), role-based access control
Resume parsing	`unpdf` text extraction + heuristic field and skill extraction, run on the server

Layer	Choice
Quality	Vitest + Supertest (backend), Vitest + Testing Library (frontend), OpenAPI 3 / Swagger UI
DevOps	Docker Compose, multi-stage Dockerfiles (nginx for the SPA), GitHub Actions CI

Security & privacy

- Passwords are hashed with bcrypt. The 8-hour JWT sits in an httpOnly, SameSite=Lax cookie. Login is rate-limited, and helmet plus a CORS allow-list are on.
- Every input is validated with Zod. The candidate profile schema is `strict()`, so fields like `email` can't be changed through it.
- All authorization goes through `lib/access.ts`. Hiring managers are scoped to their own jobs inside the SQL queries.
- Uploads are limited by MIME type and size. Files are never streamed through the API; they're served only through 5-minute presigned URLs, and every staff resume view is audited.
- Interviewers see masked email and phone numbers.
- Deletion anonymizes personal data and deletes the files, but keeps application rows, so the analytics stay correct.
- No secrets are in Git: `.env` is ignored and `.env.example` documents every variable. Real resumes are gitignored.

Testing

Suite	Result
Backend: unit + integration (Vitest + Supertest on a real Postgres test DB)	**67 / 67 passing**, 74% line coverage
Frontend: component + unit (Vitest + Testing Library)	**7 / 7 passing**

```
docker exec proeducateintern-db-1 createdb -U ats ats_test
```

```
cd backend && npm test
```

```
cd frontend && npm test
```

The tests cover authentication, RBAC (403s and IDOR), validation, the job status machine, pipeline moves, approval gates, stage-editing rules, double-booking, scorecards, export, deletion, retention, uploads and the audit log. Full breakdown: docs/TESTING.md. CI runs both suites on every push.

API

62 operations under `/api/v1`, documented in OpenAPI 3 and served by Swagger UI at **http://localhost:4000/api/docs**. Endpoint list, auth details and curl examples: docs/API.md.

Project structure

```
backend/
  prisma/          schema (21 models), migrations, seed, resume import pipeline
  src/lib/         access, auth, pipeline engine, governance, mailer, storage, resume parser, audit
  src/routes/      auth, jobs, candidate, applications, interviews, reports, admin, users
  tests/           unit + integration
frontend/
  src/pages/       landing, job board, auth, candidate portal, staff app
  src/components/  design system, layouts, match ring, theme toggle
docs/              WORKFLOW, TESTING, API, DATABASE, PHASES, screenshots, project report PDF
```

Documentation

Document	What's inside
Project report (PDF)	Problem, architecture, a page-by-page walkthrough with screenshots, security, tests, limitations
docs/WORKFLOW.md	Codebase workflow with diagrams: request lifecycle, pipeline engine, governance
docs/TESTING.md	Test workflow, every test file, requirement-to-test traceability
docs/API.md	Endpoint list, auth, errors, curl examples
docs/DATABASE.md	ER diagram, constraints, indexes, seed data
docs/PHASES.md	Build phases and the bottleneck analysis of similar systems

Submission checklist

- [x] Working, responsive web app with the complete user workflow
- [x] GitHub repo with frontend, backend, migrations, README, `.env.example`, no secrets
- [x] API documentation (OpenAPI/Swagger + docs/API.md)
- [x] Database design (ER diagram, constraints, seed data)
- [x] Unit and integration tests (67 backend + 7 frontend)
- [x] Docker setup and documented commands

- [x] Project report (PDF)
- [ ] Demo video (5–10 min)